Unit 3

Sequence in Python ¶

In Python, a sequence is a collection of objects ordered by their position. Sequences allow you to store multiple values in an organized way, where each item can be accessed by its index. Common types of sequences include lists, tuples, and strings. Sequences share several operations, such as indexing, slicing, and iteration, and many built-in functions like len(), min(), and max() work with all sequence types.

List

Lists are one of the most versatile data structures in Python. They are mutable, which means they can be modified after creation. A list can contain items of different types, including other lists.

Basic Operations

Creating a List: You can create a list by enclosing elements in square brackets [], separated by commas.

In [1]:
```
my_list = [1, 2, 3, 4, 5]
print("Initial list:", my_list)
print("ID of list:", id(my_list))
```

```
Initial list: [1, 2, 3, 4, 5]
ID of list: 2318581884480
```

This code initializes a list with five integers. The print statements display the contents of the list and the memory address (ID) of the list object.

Adding Elements

Append: Adds an item to the end of the list.

In [2]:
```
my_list.append(6)
print("List after appending 6:", my_list)
print("ID after appending:", id(my_list))
```

```
List after appending 6: [1, 2, 3, 4, 5, 6]
ID after appending: 2318581884480
```

The append method adds an item to the end of the list. The ID of the list remains unchanged, demonstrating that the list is mutable.

Inserting an Element

In [3]:
```
my_list.insert(0, 0)
print("List after inserting 0 at index 0:", my_list)
print("ID after inserting:", id(my_list))
```

```
List after inserting 0 at index 0: [0, 1, 2, 3, 4, 5, 6]
ID after inserting: 2318581884480
```

The insert method inserts an item at a specified position. The ID remains unchanged, confirming the list's mutability.

Removing an Element

In [4]:
```
my_list.remove(3)
print("List after removing element '3':", my_list)
print("ID after removing an element:", id(my_list))
```

```
List after removing element '3': [0, 1, 2, 4, 5, 6]
ID after removing an element: 2318581884480
```

The remove method removes the first occurrence of a specified value. Again, the ID remains constant.

Popping an Element

In [5]:
```
popped_element = my_list.pop()
print("Popped element:", popped_element)
print("List after popping the last element:", my_list)
print("ID after popping an element:", id(my_list))
```

```
Popped element: 6
List after popping the last element: [0, 1, 2, 4, 5]
ID after popping an element: 2318581884480
```

The pop method removes and returns an item from a specified index (default is the last item). The list's ID is unchanged.

Indexing and Slicing

Indexing

In [6]:
```
first_element = my_list[0]
print("First element:", first_element)
```

```
First element: 0
```

Indexing allows you to access an item at a particular position. Here, my_list[0] retrieves the first element.

Slicing

In [7]:
```
sub_list = my_list[1:4]
print("Sliced list from index 1 to 3:", sub_list)
print("ID of the sliced list:", id(sub_list))
```

```
Sliced list from index 1 to 3: [1, 2, 4]
ID of the sliced list: 2318581332288
```

Slicing creates a new list containing elements from a specified range. The ID of the sliced list is different, indicating that slicing creates a new list.

Built-In Functions

Length of the List

In [8]:
```
length = len(my_list)
print("Length of the list:", length)
```

```
Length of the list: 5
```

Maximum and Minimum

In [9]:
```
max_value = max(my_list)
min_value = min(my_list)
print("Maximum value:", max_value)
print("Minimum value:", min_value)
```

```
Maximum value: 5
Minimum value: 0
```

Sorting a List

The sort method sorts the list in place and does not change the ID, showing that the same list object is modified.

In [10]:
```
my_list.sort()
print("Sorted list:", my_list)
print("ID of sorted list:", id(my_list))
```

```
Sorted list: [0, 1, 2, 4, 5]
ID of sorted list: 2318581884480
```

Accessing Values in Lists

To access values in a list, use the index of the item enclosed in square brackets. Python lists are zero-indexed.

In [11]:
```
my_list = ['apple', 'banana', 'cherry']
print(my_list[1])  # Accesses the second element 'banana'
```

```
banana
```

Updating Values in Lists

Values in a list can be updated by assigning a new value to a specific index.

In [12]:
```
my_list[1] = 'orange'  # Changes 'banana' to 'orange'
print(my_list)
```

```
['apple', 'orange', 'cherry']
```

Nested Lists

A nested list is a list within another list. It is used to store complex data structures like matrices.

Accessing Elements in Nested Lists

Elements in nested lists are accessed by consecutive index operations, each index separated by its own set of square brackets.

In [13]:
```
nested_list = [1, 2, ['apple', 'banana']]
print(nested_list[2][1])  # Accesses 'banana' from the nested list

print(nested_list[2][0])  # Outputs 'apple'
```

```
banana
apple
```

Cloning Lists

Cloning or copying lists can be done with the slice operator or the list() constructor to avoid modifying the original list when changing the clone.

In [14]:
```
clone_list = my_list[:]
print(clone_list)
```

```
['apple', 'orange', 'cherry']
```

Basic List Operations

Lists support operations like concatenation, repetition, membership testing, and iteration.

In [15]:
```
# Concatenation
concatenated_list = my_list + ['mango', 'grape']
print(concatenated_list)

# Repetition
repeated_list = [0] * 3
print(repeated_list)

# Membership Testing
print('apple' in my_list)
```

```
['apple', 'orange', 'cherry', 'mango', 'grape']
[0, 0, 0]
True
```

List Methods

Python lists have various methods such as append(), extend(), pop(), reverse(), and sort() among others.

In [16]:
```
# Append and Extend
my_list.append('fig')
my_list.extend(['plum', 'pear'])
print(my_list)

# Pop
print(my_list.pop())  # Removes the last item 'pear'
```

```
['apple', 'orange', 'cherry', 'fig', 'plum', 'pear']
pear
```

Using List as Stack and Queue

Lists can be used as stacks where the last element added is the first to be removed (LIFO). For queues, use collections.deque since it is optimized for fast fixed-time operations.

In [17]:
```
stack = []
stack.append('a')
stack.append('b')
stack.pop()

from collections import deque
queue = deque(['a', 'b', 'c'])
queue.append('d')
print(queue.popleft())  # 'a'
```

```
a
```

Using Lists in Loops

Lists can be iterated over in loops directly by element or through indices

In [18]:
```
for fruit in my_list:
    print(fruit)

for i in range(len(my_list)):
    print(i, my_list[i])
```

```
apple
orange
cherry
fig
plum
0 apple
1 orange
2 cherry
3 fig
4 plum
```

1. filter Function

The filter function in Python is used to create a list (or another iterator) of elements for which a function returns true. The function takes two parameters: a function and an iterable, and it applies the function to each element of the iterable.

Explanation: filter filters elements from an iterable by applying a function that returns either True or False. If the function returns True, the element from the iterable is included in the result.

In [19]:
```
def is_positive(num):
    return num > 0

numbers = range(-5, 5)  # creates a list from -5 to 4
positive_numbers = list(filter(is_positive, numbers))
print(positive_numbers)
```

```
[1, 2, 3, 4]
```

This code defines a function is_positive that checks if a number is positive. The filter function then uses this to filter out positive numbers from a range of numbers from -5 to 4.

2. map Function

The map function applies a specified function to each item of an iterable (list, tuple etc.) and returns a list of the results.

Explanation: map is used for transforming a list by applying a function to all its elements and producing a new list with the results.

In [20]:
```
def square(num):
    return num ** 2

numbers = [1, 2, 3, 4, 5]
squared_numbers = list(map(square, numbers))
print(squared_numbers)
```

```
[1, 4, 9, 16, 25]
```

In this example, map applies the square function to each element in the list numbers. The result is a new list of the squares of these numbers.

3. reduce Function

The reduce function in Python is used for performing some computation on a list and returning the result. It applies a rolling computation to sequential pairs of values in a list.

Explanation: To use reduce, you must import it from the functools module. It is used to apply a particular function passed in its argument to all of the list elements mentioned in the sequence passed along.

In [21]:
```
from functools import reduce

def add(x, y):
    return x + y

numbers = [1, 2, 3, 4, 5]
result = reduce(add, numbers)
print(result)
```

```
15
```

This function repeatedly applies the add function to the elements of the list numbers. It starts by adding the first two (1 + 2), then adds the result to the next element (3 + 3), and continues until one single value is left, which is the sum of all elements.

Tuples

A tuple is an immutable sequence in Python, typically used to store collections of heterogeneous data. Tuples are similar to lists but are immutable, meaning they cannot be modified after creation.

Creating Tuple

In [22]:
```
my_tuple = (1, 'apple', 3.14)
print(my_tuple)
```

```
(1, 'apple', 3.14)
```

This creates a tuple containing an integer, a string, and a float. The tuple is printed directly. Utility of Tuples Tuples are faster than lists and protect the data, which is permanent and should not be changed. Tuples are commonly used for small collections of values that will not need to change, such as an IP address and port.

Accessing Values in a Tuple

In [23]:
```
print(my_tuple[1])  # Accesses the second element 'apple'
```

```
apple
```

Accessing the tuple by index retrieves the value at that position.

Updating Values in a Tuple

Tuples are immutable; hence they do not allow updates directly. However, you can work around this by converting the tuple to a list, changing the list, and converting it back to a tuple.

In [24]:
```
temp_list = list(my_tuple)
temp_list[1] = 'banana'
my_tuple = tuple(temp_list)
print(my_tuple)
```

```
(1, 'banana', 3.14)
```

Deleting Elements in Tuple

Since tuples are immutable, elements cannot be deleted individually. You can delete an entire tuple though.

In [25]:
```
del my_tuple
```

Basic Operations on Tuples

Operations like concatenation and repetition are allowed, as they produce a new tuple.

In [26]:
```
t1 = (1, 2, 3)
t2 = (4, 5, 6)
print(t1 + t2)  # Concatenation
print(t1 * 3)   # Repetition
```

```
(1, 2, 3, 4, 5, 6)
(1, 2, 3, 1, 2, 3, 1, 2, 3)
```

Tuple Assignment

Tuple assignment allows simultaneous assignment of values from a tuple.

In [27]:
```
(x, y, z) = t1
print(x, y, z)
```

```
1 2 3
```

Tuples Returning Multiple Values

Functions can return tuples to return multiple values.

In [28]:
```
def min_max(numbers):
    return min(numbers), max(numbers)
print(min_max([1, 2, 3, 4, 5]))
```

```
(1, 5)
```

Nested Tuple

Tuples can contain other tuples as elements.

In [29]:
```
nested_tuple = (1, 2, (3, 4))
print(nested_tuple[2][0])
```

```
3
```

Checking the Index: index() Method

Finds the first occurrence of a value.

In [30]:
```
print(t1.index(2))  # Outputs: 1
```

```
1
```

Counting the Elements count()

Counts how many times an element appears.

In [31]:
```
print(t1.count(2))  # Outputs: 1
```

```
1
```

List Comprehension and Tuple

You can use list comprehension to create a tuple by generating a list first.

In [32]:
```
tuple_from_list = tuple([x * 2 for x in range(5)])
print(tuple_from_list)
```

```
(0, 2, 4, 6, 8)
```

Variable Length Argument Tuples

Used in function definitions to allow for an arbitrary number of arguments.

In [33]:
```
def func(*args):
    return args
print(func(1, 2, 3))
```

```
(1, 2, 3)
```

The zip() Function

zip() is used to pair elements from two or more iterables (like tuples or lists).

In [34]:
```
result = zip((1, 2, 3), ('a', 'b', 'c'))
print(list(result))
```

```
[(1, 'a'), (2, 'b'), (3, 'c')]
```

Advantage of Tuple over List

Tuples are faster than lists, which makes a significant difference for large collections of data. Being immutable, they can be used as keys in dictionaries, which is not possible with lists.

Sets

A set is an unordered collection of unique elements. Sets are mutable and are useful when the presence of an element in a collection is more important than the order or how many times it occurs.

In [35]:
```
my_set = {1, 2, 3, 2, 1}
print(my_set)  # Output will be {1, 2, 3}
```

```
{1, 2, 3}
```

Sets automatically remove duplicates. The set will only contain unique elements.

Dictionaries

A dictionary in Python is a collection of key-value pairs, where each key is unique.

Creating Dictionaries

In [36]:
```
my_dict = {'name': 'Alice', 'age': 25}
print(my_dict)
```

```
{'name': 'Alice', 'age': 25}
```

This creates a dictionary with two keys: 'name' and 'age', and their corresponding values 'Alice' and 25.

Accessing Values

In [37]:
```
print(my_dict['name'])  # Outputs: Alice
```

```
Alice
```

Values can be accessed using their keys in square brackets.

Adding and Modifying an Item in a Dictionary

In [38]:
```
my_dict['gender'] = 'Female'  # Adds a new key-value pair
my_dict['age'] = 26  # Modifies the value of an existing key
print(my_dict)
```

```
{'name': 'Alice', 'age': 26, 'gender': 'Female'}
```

Modifying an Entry

This is similar to adding an entry; if the key exists, its value will be updated.

Deleting Item

In [39]:
```
del my_dict['gender']
print(my_dict)
```

```
{'name': 'Alice', 'age': 26}
```

Sorting Items in Dictionary

Sorting can refer to sorting the keys, values, or key-value pairs.

In [40]:
```
sorted_keys = sorted(my_dict.keys())
print(sorted_keys)  # Outputs sorted list of keys
```

```
['age', 'name']
```

Looping Over a Dictionary

In [41]:
```
for key, value in my_dict.items():
    print(f"{key}: {value}")
```

```
name: Alice
age: 26
```

Nested Dictionaries

In [42]:
```
nested_dict = {'dict1': {'key1': 'value1'}, 'dict2': {'key2': 'value2'}}
print(nested_dict['dict1']['key1'])  # Outputs: value1
```

```
value1
```

Difference Between a List and a Dictionary

Lists are ordered sets of objects, accessible by indices. Dictionaries are unordered sets of key-value pairs, accessible by keys.

List vs Tuple vs Dictionary

List: Ordered, mutable, accessed by index. Tuple: Ordered, immutable, accessed by index. Dictionary: Unordered, mutable, accessed by unique keys, good for fast lookup.

Built-in Dictionary Functions and Methods

Python dictionaries come with a variety of built-in functions and methods that make manipulating the contents of dictionaries straightforward and efficient. Below, we explore some of the most commonly used methods.

1. .keys()

The .keys() method returns a view object that displays a list of all the keys in the dictionary.

In [43]:
```
my_dict = {'name': 'Alice', 'age': 25, 'gender': 'Female'}
keys = my_dict.keys()
print(keys)
```

```
dict_keys(['name', 'age', 'gender'])
```

This will output dict_keys(['name', 'age', 'gender']), which is a view of all the keys in my_dict.

2. .values()

The .values() method returns a view object that contains a list of all the values in the dictionary.

In [44]:
```
values = my_dict.values()
print(values)
```

```
dict_values(['Alice', 25, 'Female'])
```

This will output dict_values(['Alice', 25, 'Female']), showing all the values in my_dict.

3. .items()

The .items() method returns a view of the dictionary's key-value pairs (as tuples).

In [45]:
```
items = my_dict.items()
print(items)
```

```
dict_items([('name', 'Alice'), ('age', 25), ('gender', 'Female')])
```

This will output dict_items([('name', 'Alice'), ('age', 25), ('gender', 'Female')]), displaying the dictionary in a tuple format.

4. .get()

The .get() method returns the value for the specified key if the key is in the dictionary. It allows you to provide a default value if the key does not exist.

In [46]:
```
print(my_dict.get('name'))  # Outputs: Alice
print(my_dict.get('location', 'Not Available'))  # Outputs: Not Available
```

```
Alice
Not Available
```

get() retrieves the value for 'name' and provides a default for 'location' since it isn't a key in my_dict.

5. .setdefault()

The .setdefault() method returns the value of a key (if the key is in the dictionary). If not, it inserts the key with a specified value.

In [47]:
```
my_dict.setdefault('name', 'Default Name')  # Key exists
my_dict.setdefault('location', 'Unknown')  # Key does not exist
print(my_dict)
```

```
{'name': 'Alice', 'age': 25, 'gender': 'Female', 'location': 'Unknown'}
```

Since 'name' already exists, its value remains unchanged. 'location' is added with the value 'Unknown'.

6. .pop()

The .pop() method removes and returns an element from a dictionary given a specific key.

In [48]:
```
age = my_dict.pop('age')
print(age)
print(my_dict)
```

```
25
{'name': 'Alice', 'gender': 'Female', 'location': 'Unknown'}
```

The 'age' key is removed from my_dict, and its value 25 is printed. The remaining dictionary is printed without the 'age' key.

7. .popitem()

The .popitem() method removes and returns the last inserted key-value pair from the dictionary.

In [49]:
```
item = my_dict.popitem()
print(item)
print(my_dict)
```

```
('location', 'Unknown')
{'name': 'Alice', 'gender': 'Female'}
```

Removes the last key-value pair added to my_dict and prints it, along with the modified dictionary.